Database Development and Design (SOE201)

Lab 2: Practical on JDBC and Index

Dr. Wei Wang

AIAT

Practical on JDBC and Index

- This lab introduces Java Database Connectivity (JDBC) and programming with index.
  - Task 1:
    - 1.1: set up all environment; understand and test the given code (*JDBC.java*) on basic JDBC query.
    - 1.2: understand and test the given code (*JDBCandIndex.java*) on basic JDBC query with an index.
  - Task 2: to be released during lab.

Preparation on Using MySQL Server

MySQL Database

- To use a database one can either:
  - Download and install MySQL on one's own computer, or
  - Use school's MySQL server.
    - Read attached document '*new_MySQL#1 Create privilege.pdf*';
    - http://csse-mysql.xjtlu.edu.cn
    - Default password is "SOE201". Change password after first login (required by school)

Install MySQL

- Download and install MySQL:
  - https://dev.mysql.com/downloads/mysql/
- Download MySQL JDBC API (middleware)
  - https://dev.mysql.com/downloads/connector/j/
  - Install it in classpath (or import it in your IDE)

Use School's MySQL

- Create a database and relations (tables) on school's MySQL server.
  - SAT MySQL: http://csse-mysql.xjtlu.edu.cn (IP: *10.7.1.127, only available on campus network)*
- Setup XJTLU VPN (*off campus access only*)
  - guide.xjtlu.edu.cn->IT Guide for Staff/Student(at top of the webpage)->STAFF/Student Guide to IT service.pdf.
  - **May NOT support now.**

Use School's MySQL – cont'd

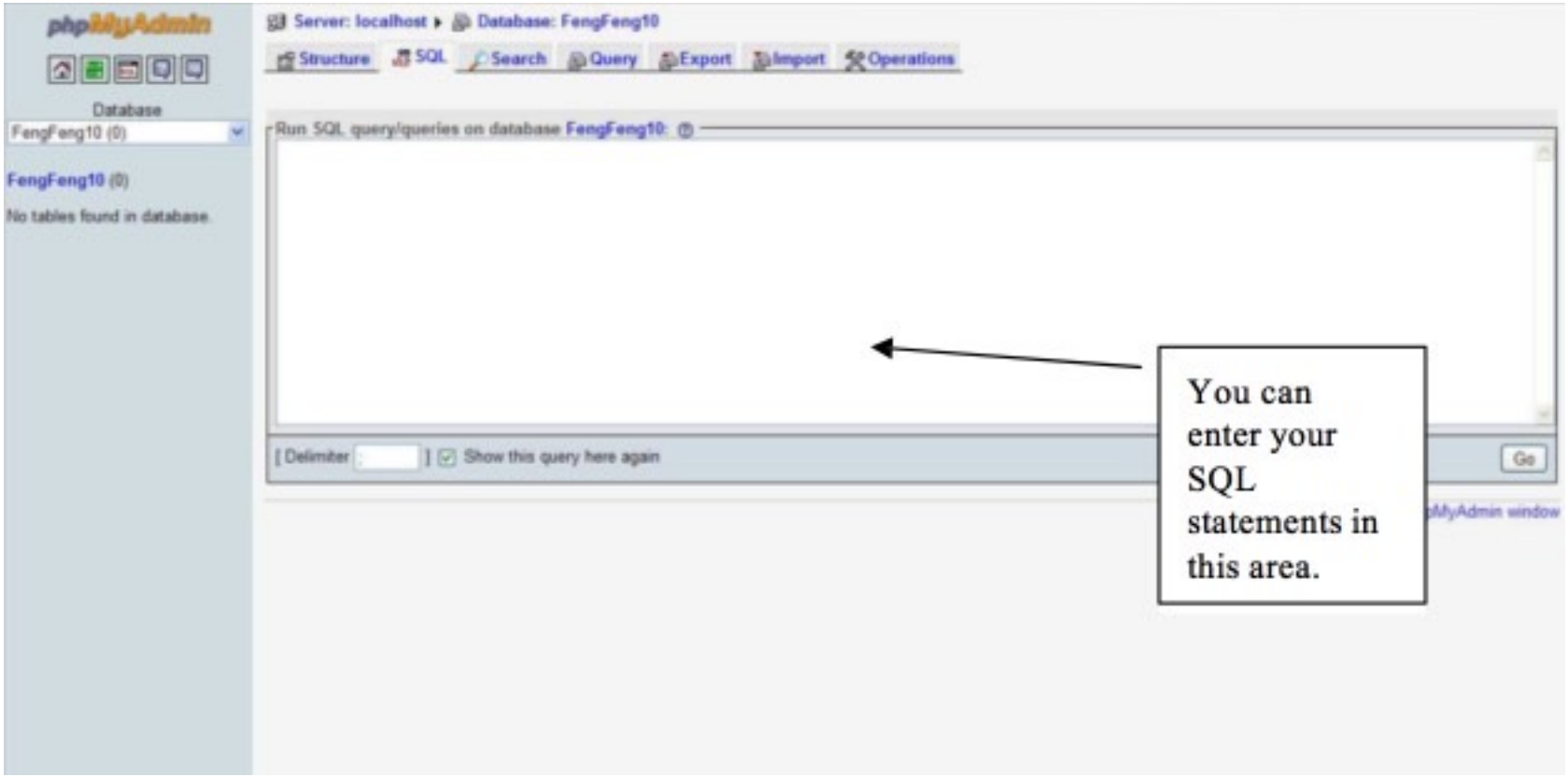
- No need to explicitly create database as it is created after login.



Xi'an Jiaotong-Liverpool University 西交利物浦大学

Use School's MySQL – cont'd

- Click on SQL tab or SQL icon and create tables.



Xi'an Jiaotong-Liverpool University 西交利物浦大学

Database Preparation

- Once logged in with one's username and password, system will automatically create a database having same name as one's username.
- Importing file '*yeast_prosite.sql*' into database. The document includes scripts to:
  - Create tables
  - Populate tables, i.e. inserting records

Connect MySQL with JDBC

- READ "quick tutorial on JDBC" below.
- MySQL provides connectivity for client applications developed in Java through JDBC driver, which is called MySQL Connector/J.
- MySQL Connector/J is a JDBC Type 4 driver. Type 4 designation means that driver is a pure Java implementation of MySQL protocol and does not rely on MySQL client libraries.
  - For large-scale programs that use common design patterns of data access, consider using one of popular persistence frameworks such as Hibernate or MyBatis.
- Download SQL java connector:
  - https://dev.mysql.com/downloads/connector/j/
  - Current version Connector/J 9.4.0; older version also work

Preparation

- Download files from LM Core
  - *JDBCIndex.java*
  - *Needs to be modified with one's own parameters.*
    - *Package name*
    - *Connection parameters*
- Open the *JDBCIndex.java* in any editor. Replace following three parameters with your own ones.
  - *static final String DB_URL = "jdbc:mysql://10.7.1.36/[DATABASE_NAME]";*
  - *static final String USER = "[USERNAME]";*
  - *static final String PASS = "[PASSWORD]";*

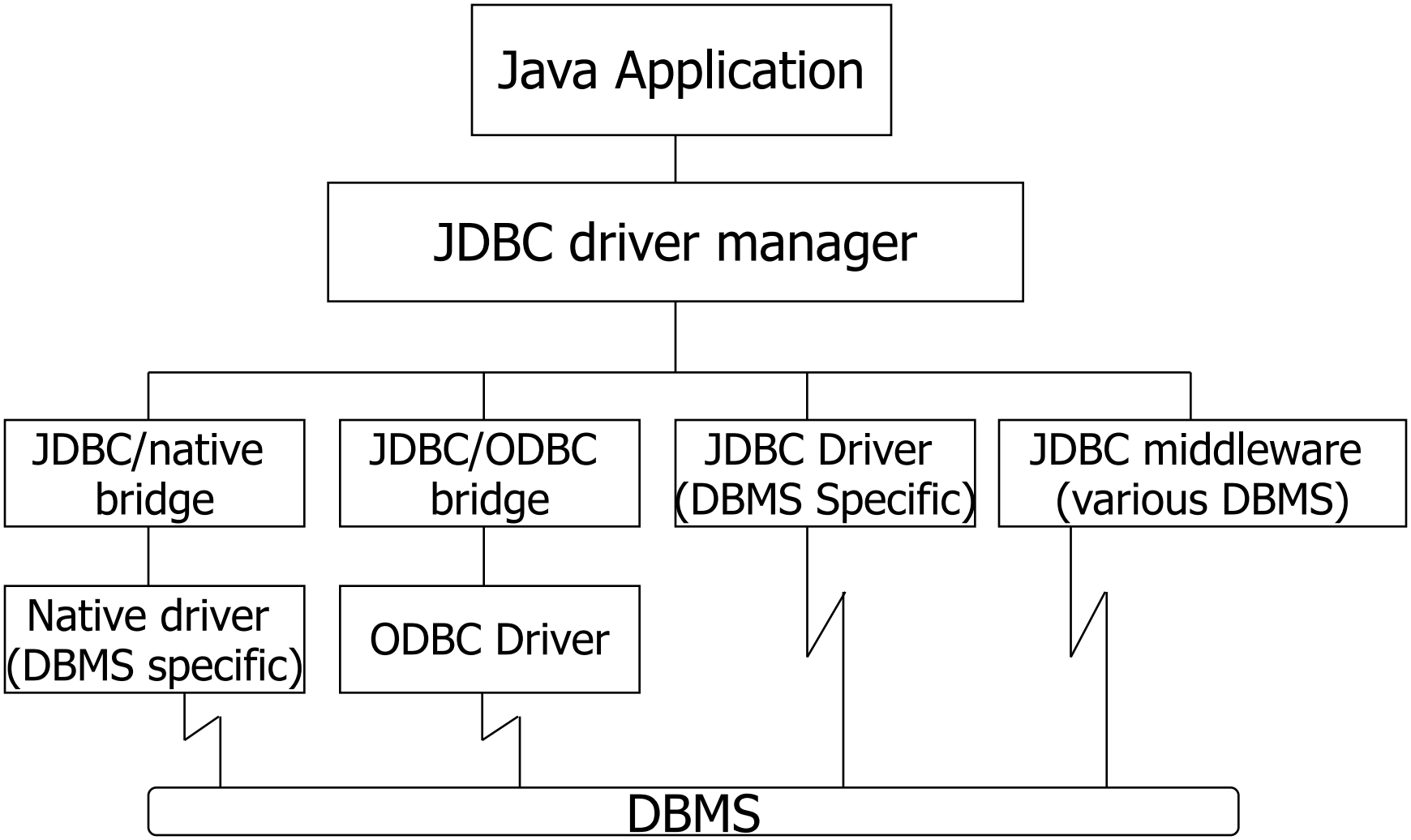
Quick JDBC Tutorial

What is JDBC?

- JDBC is short for Java Database Connectivity
  - Java API for connecting programs written in Java to data in relational databases.
  - Consists of a set of classes and interfaces written in Java programming language.
  - Provides a standard API for tool/database developers and makes it possible to write database applications using pure Java API.
  - Defined by Sun Microsystems, allowing individual providers to implement and extend standard with their own JDBC drivers.

Xi'an Jiaotong-Liverpool University 西交利物浦大学

JDBC Architectures



Xi'an Jiaotong-Liverpool University 西交利物浦大学

JDBC API Overview

- JDBC functionalities
  - Establish a connection with a database or access any tabular data source
  - Send SQL statement
  - Process result
- JDBC components
  - DriverManager and Driver
  - Connection
  - Statement
  - ResultSet
  - SQLException
  - etc

Xi'an Jiaotong-Liverpool University 西交利物浦大学

Creating JDBC Application

- 1. Import necessary packages
- 2. Register JDBC driver
- 3. Establish connection
- 4. Execute query
- 5. Extract and intepret data from result set
- 6. Clean up environment

Step 1: Import Packages

- Include packages containing JDBC classes needed for database programming.
- Example using MySQL
  - import java.sql.Connection;
  - import java.sql.DriverManager;
  - import java.sql.ResultSet;
  - import java.sql.SQLException;
  - import java.sql.Statement;

Step 2: Register JDBC Driver

- Initialise driver, in order to establish communication channel with database
- Example using MySQL
  - Class.forName("com.mysql.cj.jdbc.Driver");

Step 3: Establish a Connection

- Create Connection object, which represents physical connection with database
  - static final String USER = "your_username";
  - static final String PASS = "your_password";
  - static final String DB_URL = "your_url";
  - conn = DriverManager.getConnection(DB_URL,USER,PASS);

Step 4: Execute a Query

- Use Statement (or PreparedStatement or CallableStatement) for constructing and submitting SQL QUERY statement to database
  - stmt = conn.createStatement();
  - String sql = "SELECT id, name, hometown FROM student";
  - ResultSet rs = stmt.executeQuery(sql);
- For UPDATE, INSERT or DELETE statement
  - stmt = conn.createStatement();
  - String sql = "DELETE FROM student WHERE id = 1";
  - int num = stmt.executeUpdate(sql);

Step 4: Execute a Query - cont'd

- Send query (SELECT) to database
  - executeQuery: returns one ResultSet object
- Send update (INSERT, DELETE, UPDATE) to database
  - executeUpdate: returns an integer representing number of rows affected by SQL statement

Step 5: Extract and Interpret Data from ResultSet

- Use getXXX() to retrieve data from ResultSet object

```
while(rs.next()){
//Retrieve by column name
      int id = rs.getInt("id");
      String name = rs.getString("name");
      String hometown = rs.getString("hometown");
}
```

Step 6: Clean Up Environment

- Explicitly close all database resources
  - rs.close();
  - stmt.close();
  - conn.close();

Mapping Data Types

- There are data types specified to SQL that need to be mapped to Java data types
  - If users expect Java to be able to handle them.
- Conversion falls into three categories:
  - SQL type to Java direct equivalents
    - SQL INTEGER direct equivalent of Java int data type
  - SQL type can be converted to a Java equivalent
    - SQL CHAR, VARCHAR, and LONGVARCHAR can all be converted to Java String data type
  - SQL data type is unique and requires a special Java class object to be created specifically for their SQL equivalent
    - SQL DATE converted to Java Date object that is defined in java.Date for this purpose

Xi'an Jiaotong-Liverpool University 西交利物浦大学

Data Types and Methods

SQL	*JDBC/Java*	*setXXX()*	*updateXXX()*
VARCHAR	java.lang.String	setString()	updateString()
CHAR	java.lang.String	setString()	updateString()
NUMERIC	java.math.BigDecimal	setBigDecimal()	updateBigDecimal()
INTEGER	int	setInt()	updateInt()
FLOAT	float	setFloat()	updateFloat()
DOUBLE	double	setDouble()	updateDouble()
DATE	java.sql.Date	setDate()	updateDate()
TIME	java.sql.Time	setTime()	updateTime()
TIMESTAMP	java.sql.Timestamp	setTimestamp()	updateTimestamp()

Three Types of Statements

Interface	*Recommended Use*
Statement Used to implement simple SQL statements with no parameters.	• Use for general-purpose access to database. • Useful with static SQL statements at runtime. • Statement interface cannot accept parameters.
PreparedStatement (Extends Statement) Used for precompiling SQL statements that might contain input parameters.	• Use when plan to execute SQL statements many times. • PreparedStatement interface accepts input parameters at runtime.
CallableStatement (Extends PreparedStatement) Used to execute stored procedures or functions that may contain both input and output parameters.	• Use when access database stored procedures. • CallableStatement interface can also accept runtime input parameters.